

CS214 Project 2 Final Report

Hugo Wan, GitHub ID: HugoWan0504

Deadline: 6/8/2026

1 Introduction

In this project, I implemented and optimized a parallel Breadth-First Search (BFS) algorithm in `BFS.h`. The goal of the project is to compute the BFS distance from a source vertex s to every other reachable vertex in a graph. The input graph is stored in compressed sparse row (CSR) format using the arrays `offsets` and `edges`, and the output distances are stored in the array `dist`.

The final grading keeps only `BFS.h`, so I kept all implementation logic inside this file. My final version is Optimization 11, which uses graph-adaptive BFS strategies. For high-degree social graphs such as `com-orkut`, it uses sparse/dense direction switching. For lower-degree social graphs such as `soc-LiveJournal1`, it uses a different set of thresholds to reduce parallel overhead. For `RoadUSA`, which is a larger-diameter road-like graph, it avoids dense scanning and uses a sparse frontier strategy.

Important grading note. The final implementation to grade is the `BFS.h` file located in the root directory of my GitHub repository. The other optimization folders, such as `baseline/`, `Opt1-5/`, `Opt6-10/`, and `Opt11/`, are included only to show the development history and should not be used as the final submitted code. The root-level `BFS.h` contains my final Opt11 implementation.

2 Baseline Implementation

The baseline implementation was a normal sequential BFS using `std::queue`. It starts from the source vertex, repeatedly pops one vertex from the queue, scans its neighbors, and pushes newly discovered vertices into the queue.

The baseline logic is simple and correct, but it is not parallel. It processes one vertex at a time, so it cannot take advantage of the fact that all vertices in the same BFS level can be processed independently. This baseline was useful as a correctness reference and as the timing baseline for computing speedups.

3 Final Algorithm Design

My final implementation is based on a frontier-based BFS structure. Instead of using one queue, it maintains two arrays:

- **frontier**: the current BFS level.
- **next_frontier**: the vertices discovered for the next BFS level.

Each BFS round processes all vertices in the current frontier and builds the next frontier. This matches the level-by-level structure of BFS.

3.1 Sparse Top-Down Mode

In sparse mode, the algorithm scans the neighbors of vertices in the current frontier. For each frontier vertex u , it checks every neighbor v . If v has not been visited, the algorithm updates `dist[v]` and adds v to the next frontier.

For parallel sparse mode, multiple frontier vertices may try to discover the same neighbor. To avoid duplicate discovery, I used atomic compare-and-swap:

`CAS(dist[v], INT_MAX, dist[u] + 1)`

This ensures that only one worker successfully claims each unvisited vertex.

3.2 Dense Bottom-Up Mode

For large social-graph frontiers, sparse mode can become expensive because it scans many outgoing edges from the frontier. To handle this, I added dense bottom-up BFS. In dense mode, the algorithm marks the current frontier in a boolean array `in_frontier`. Then it scans unvisited vertices and checks whether any neighbor belongs to the current frontier. If so, that vertex is discovered for the next level.

This is useful for social graphs because large frontiers appear frequently. Instead of pushing from a huge frontier, dense mode pulls from unvisited vertices and can stop early once it finds a frontier neighbor.

3.3 Dense Frontier Compaction

A major improvement came from changing how the dense mode builds the next frontier. Earlier versions used atomic increments to push discovered vertices directly into `next_frontier`. This created many atomic operations.

In the final optimized dense mode, each discovered vertex is first marked in a temporary boolean array `next_flag`. Then the algorithm compacts the marked vertices into `next_frontier`. This compaction is done block by block:

1. Count discovered vertices in each block.
2. Compute block offsets.

3. Fill `next_frontier` in parallel using the block offsets.

This reduced the number of atomic operations in dense mode and significantly improved performance on the social graphs.

4 Graph-Adaptive Strategy

One important observation was that one BFS strategy did not work equally well for all three graphs. The social graphs and the road graph have very different structures.

4.1 High-Degree Social Graph: `com-orkut`

The `com-orkut` graph has a very high average degree. Large frontiers appear quickly, so sparse/dense switching works very well. For this graph-like case, I used:

`DENSE_FRONTIER_THRESHOLD = n / 50`

and block size:

`BLOCK_SIZE = 4096`

This gave strong performance while keeping the implementation simple.

4.2 Lower-Degree Social Graph: `soc-LiveJournal1`

The `soc-LiveJournal1` graph has lower average degree than `com-orkut`. During tuning, I found that the same parameters were not always best. The final version uses a separate LiveJournal-like branch with:

`DENSE_FRONTIER_THRESHOLD = n / 70`

`BLOCK_SIZE = 2048`

`SEQ_FRONTIER_THRESHOLD = 131072`

The larger sequential sparse threshold was useful because many sparse frontier rounds did not have enough work to justify parallel overhead. Processing more small and medium frontiers sequentially improved the LiveJournal running time.

4.3 Road-Like Graph: `RoadUSA`

The `RoadUSA` graph has much larger diameter and lower average degree. Dense bottom-up BFS is not effective here because scanning all vertices for many BFS rounds becomes too expensive. Therefore, the final version avoids dense mode for RoadUSA-like graphs.

For RoadUSA, I used a sparse frontier BFS with a threshold:

ROAD_PAR_THRESHOLD = 131072

Small frontiers are processed sequentially, and parallel sparse processing is only used when the frontier is large enough. This gives a modest improvement over the baseline without hurting the social graph performance.

5 Optimization Progress

I developed the implementation gradually from the baseline to Optimization 11. The main purpose was to test one idea at a time and compare the effect on each graph.

- **Baseline:** Sequential queue-based BFS.
- **Opt 1:** Replaced the queue with two frontier arrays and used parallel frontier processing with atomic compare-and-swap.
- **Opt 2:** Added a coarsening threshold so small frontiers are processed sequentially.
- **Opt 3:** Added dense bottom-up mode for large frontiers.
- **Opt 4:** Replaced atomic dense-mode pushes with a temporary next-frontier flag array.
- **Opt 5:** Made dense-mode compaction block-parallel.
- **Opt 6–Opt 9:** Added graph-adaptive behavior and RoadUSA-specific handling.
- **Opt 10–Opt 11:** Tuned the LiveJournal-like branch, especially the dense threshold, block size, and sequential sparse threshold.

6 Testing Results

I tested the implementations on the three graph files used in the project:

- `com-orkut_sym.bin`
- `soc-LiveJournal1_sym.bin`
- `RoadUSA_sym.bin`

The average running times are shown in Table 1.

Table 1: Average running time of each implementation version.

Version	com-orkut	soc-LiveJournal1	RoadUSA
Baseline	1.49022	0.947512	1.23298
Opt 1	1.15520	0.577739	2.35318
Opt 2	0.756791	0.836324	2.44002
Opt 3	0.199320	0.347800	2.30373
Opt 4	0.072260	0.107092	2.44907
Opt 5	0.0399763	0.0607897	2.30982
Opt 6	0.047011	0.0664807	1.23635
Opt 7	0.0412737	0.0645117	1.09698
Opt 8	0.0419063	0.073641	1.04952
Opt 9	0.042463	0.0682477	1.08954
Opt 10	0.0447633	0.0614033	1.02840
Opt 11	0.0435477	0.038949	1.02668

The speedups compared with the baseline are shown in Table 2.

Table 2: Speedup of each implementation version compared with the baseline.

Version	com-orkut	soc-LiveJournal1	RoadUSA
Baseline	1.00×	1.00×	1.00×
Opt 1	1.29×	1.64×	0.52×
Opt 2	1.97×	1.13×	0.51×
Opt 3	7.48×	2.72×	0.54×
Opt 4	20.62×	8.85×	0.50×
Opt 5	37.28×	15.59×	0.53×
Opt 6	31.70×	14.25×	1.00×
Opt 7	36.11×	14.69×	1.12×
Opt 8	35.56×	12.87×	1.17×
Opt 9	35.09×	13.88×	1.13×
Opt 10	33.29×	15.43×	1.20×
Opt 11	34.22×	24.33×	1.20×

7 Analysis

The results show that sparse/dense direction switching is very effective for the two social graphs. For **com-orkut**, the speedup improved from the baseline to 34.22× in the final version. This is above the 20× speedup tier, which is the highest tier listed in the project grading table for **com-orkut**.

For **soc-LiveJournal1**, the final version reached 24.33× speedup. This was the main improvement from Opt10 to Opt11. The biggest change was increasing the sequential sparse threshold for the LiveJournal-like branch. This suggests that for lower-degree social graphs,

parallel overhead in sparse rounds can be expensive. Processing more small and medium sparse frontiers sequentially helped reduce that overhead.

For **RoadUSA**, the final speedup is only $1.20\times$, but this is still better than many of the earlier social-graph-focused versions. The earlier dense-mode optimizations were actually much slower on **RoadUSA** because the graph has larger diameter and smaller frontiers. This confirmed that **RoadUSA** needs a different strategy from the social graphs. The final **RoadUSA** path avoids dense scans and uses a sparse frontier strategy instead.

Overall, **Opt11** is the best final version because it gives strong speedups on the two required social graphs while still achieving a modest improvement on the **RoadUSA** bonus graph.

8 Correctness

The BFS implementation preserves correctness by only assigning a distance to a vertex when it is first discovered. In sparse parallel mode, atomic compare-and-swap ensures that only one thread can successfully change `dist[v]` from `INT_MAX` to the new BFS distance. This avoids duplicate discovery and prevents conflicting writes.

In dense mode, each unvisited vertex is scanned independently. If it has a neighbor in the current frontier, it is assigned the next BFS distance and marked for the next frontier. Since the algorithm processes BFS level by BFS level, this gives the correct shortest-path distance from the source.

I also used the provided checker in `BFS.cpp` during testing to verify that the produced distances matched the expected BFS results.

9 Conclusion

In this project, I implemented a parallel BFS algorithm and optimized it through multiple versions. The final implementation uses a graph-adaptive strategy. For social graphs, it uses sparse/dense direction switching and block-parallel dense frontier compaction. For **RoadUSA**, it avoids dense scans and uses a sparse frontier approach.

The final version, **Opt11**, achieved $34.22\times$ speedup on **com-orkut**, $24.33\times$ speedup on **soc-LiveJournal1**, and $1.20\times$ speedup on **RoadUSA**. These results show that the final implementation performs strongly on the required social graph tests and also gives a modest improvement on the bonus road graph.

10 Acknowledgment and References

I used the CS214 Project 2 handout as the main project specification. I also used the course lecture materials on parallel graph algorithms, compressed sparse row graph representation, frontier-based BFS, coarsening, and parallel algorithm implementation ideas. I used ChatGPT to help organize my optimization progress, explain implementation tradeoffs, and draft report text. I will attach the relevant ChatGPT conversation as required by the course policy.

References

- [1] Yihan Sun. CS214 Project 2 Handout, *Parallel BFS*, Spring 2026.
- [2] Scott Beamer, Krste Asanović, and David Patterson. *Direction-Optimizing Breadth-First Search*. Scientific Programming, 21(3–4):137–148, 2013.
- [3] Julian Shun and Guy E. Blelloch. *Ligra: A Lightweight Graph Processing Framework for Shared Memory*. Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, 2013.
- [4] Letong Wang, Xiaojun Dong, Yan Gu, and Yihan Sun. *Parallel Strong Connectivity Based on Faster Reachability*. ACM SIGMOD, 2023.
- [5] ChatGPT conversation used for implementation discussion, optimization planning, and report drafting. https://chatgpt.com/s/t_6a0a1a50b4d08191a91b49fc929e2ba3